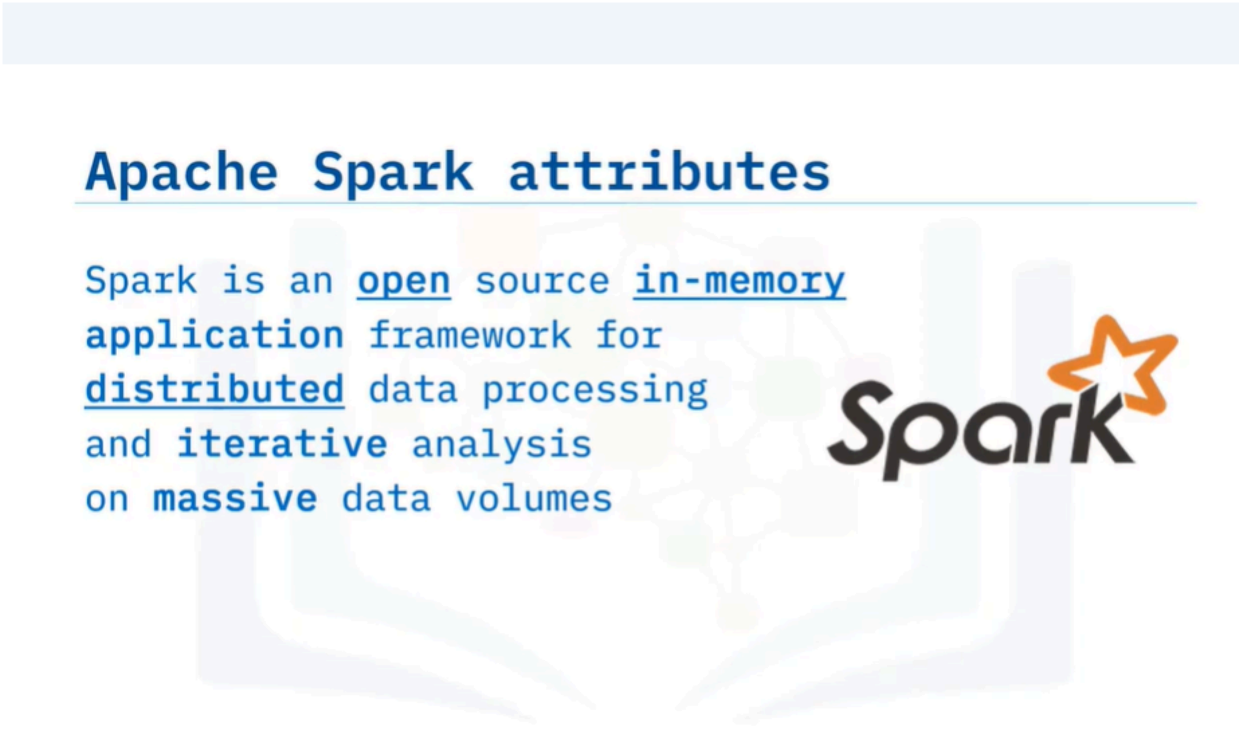
Why use Apache Spark

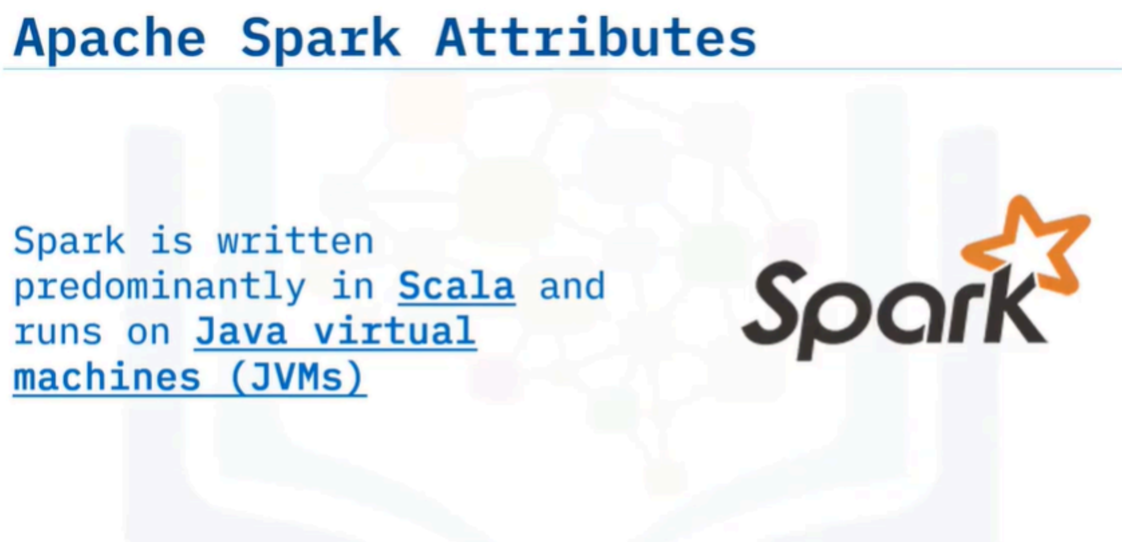
Sure! Let's dive into the concept of **Apache Spark** in simple terms.

Apache Spark is like a super-fast chef in a kitchen that can handle a lot of cooking at once. Imagine you have a huge banquet to prepare, and instead of cooking everything one dish at a time, you have a team of chefs (computers) working together. Each chef can work on their own dish simultaneously, which speeds up the entire cooking process. This teamwork is what we call **distributed computing**. In Spark, all the ingredients (data) are kept in memory (like having everything on the counter) so that the chefs can grab what they need quickly without running back and forth to the pantry (disk storage).

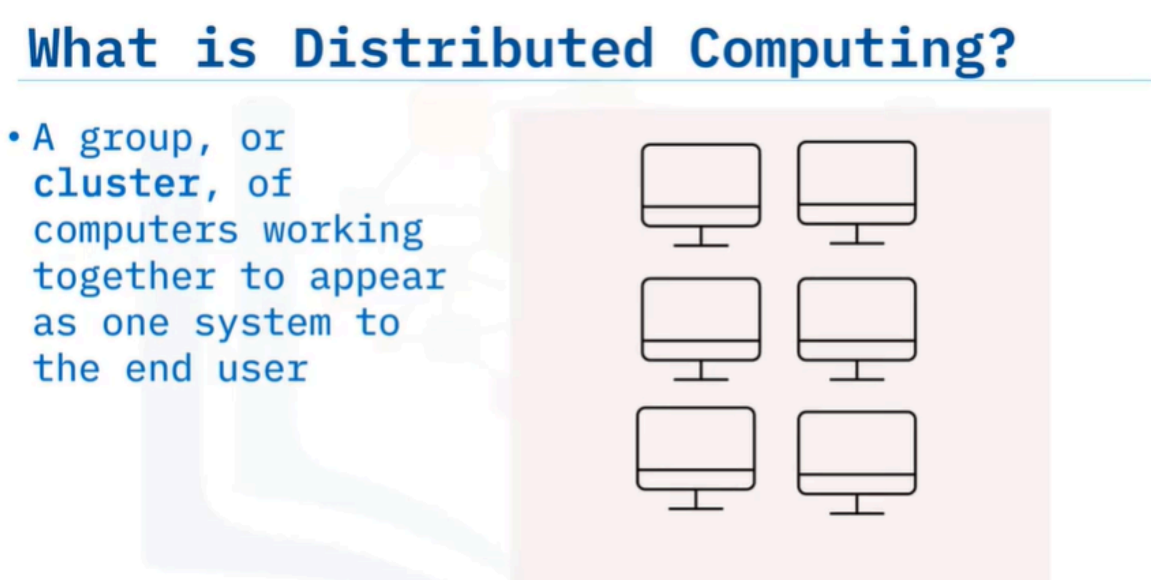
Now, think about how traditional cooking methods might require you to keep going back to the pantry to get more ingredients, which takes time. This is similar to how older systems like MapReduce work, where they have to read and write data to disk, slowing things down. Spark, on the other hand, keeps most of the data in memory, making it much faster and more efficient for handling large amounts of data.



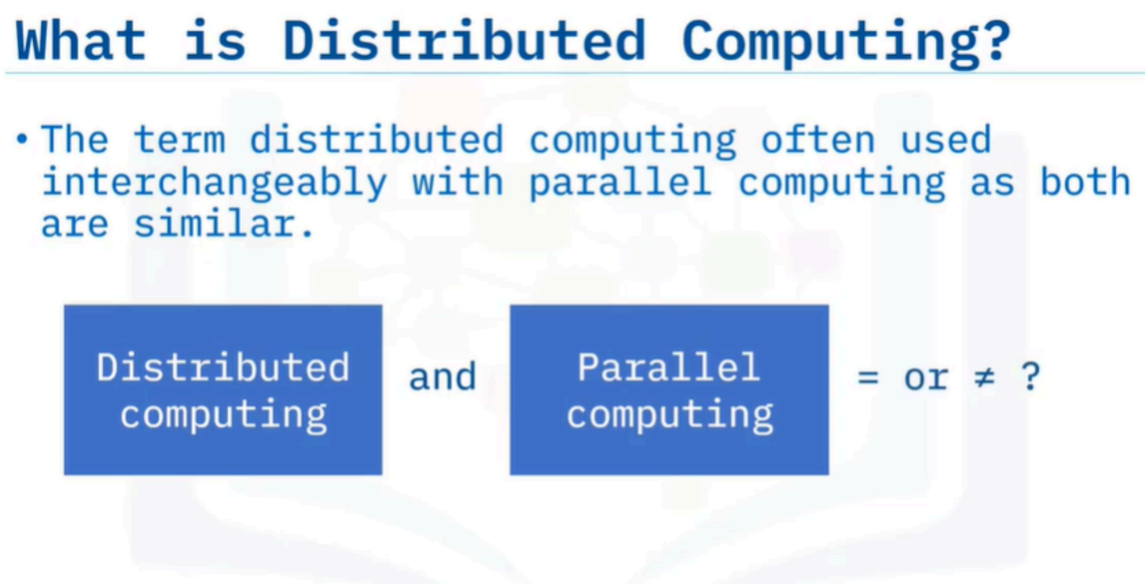
- Spark is an open-source, in-memory application framework for distributed data processing and iterative analysis on massive data volumes. Let's review the keywords in this sentence: First, Spark is entirely open source. Spark is available under the Apache Foundation hence the name Apache Spark. Next, Spark is "in-memory," which means that all operations happen within the memory or RAM. Distributed data processing uses Spark. Finally, Spark scales very well with data.



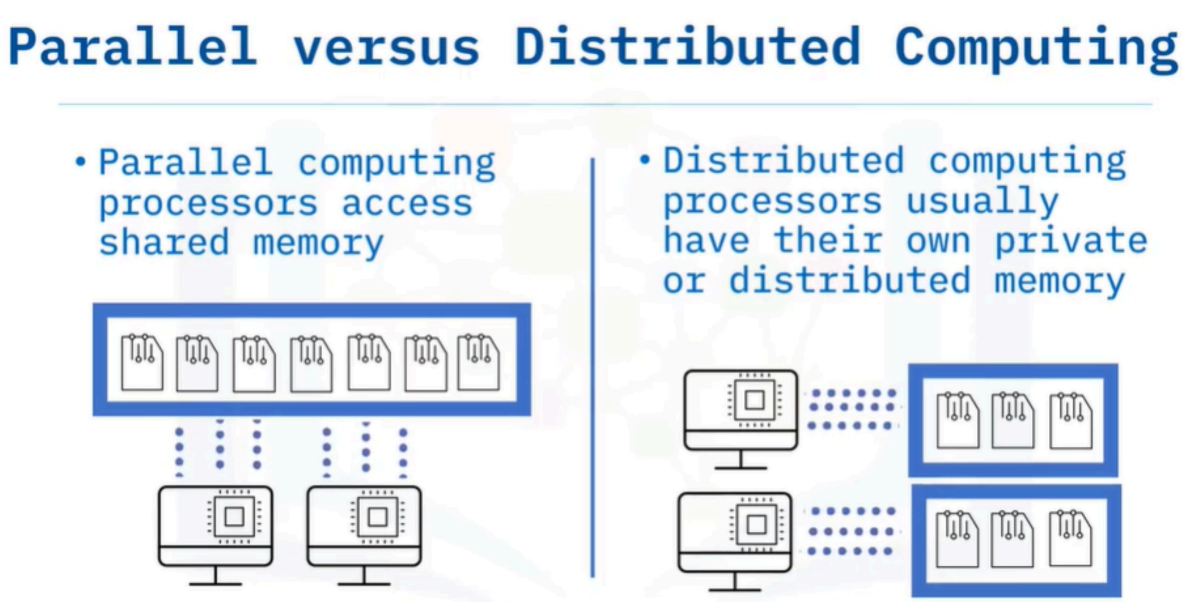
- which is ideal for massive datasets. Spark is primarily written in Scala, a general-purpose programming language that supports both object-oriented and functional programming. Spark runs on Java virtual machines.



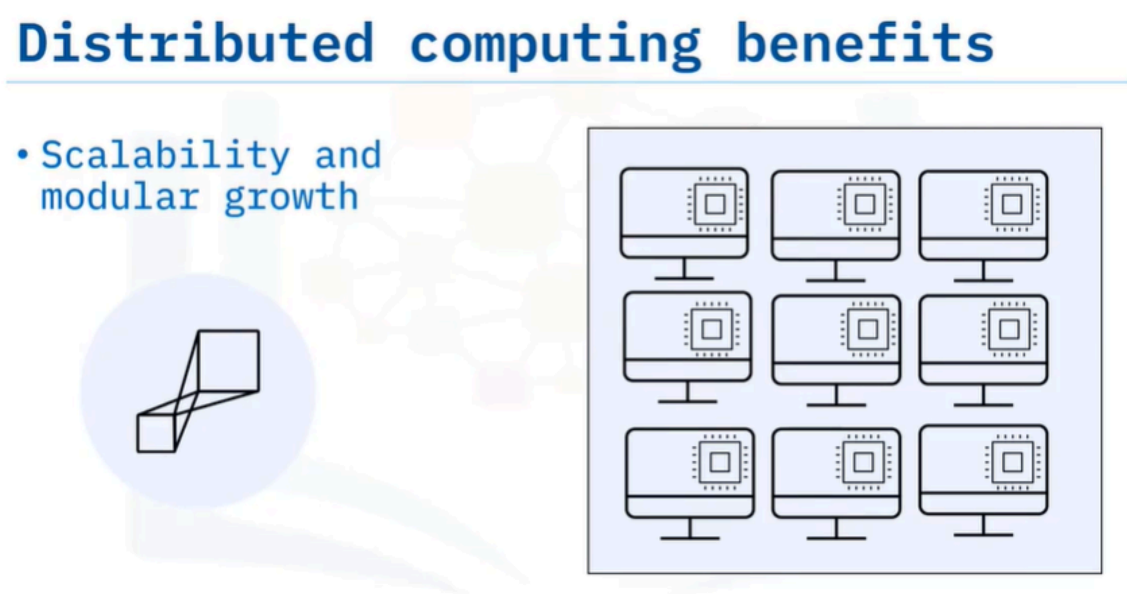
- Distributed computing, as the name suggests, is a group of computers or processors working together behind the scenes.



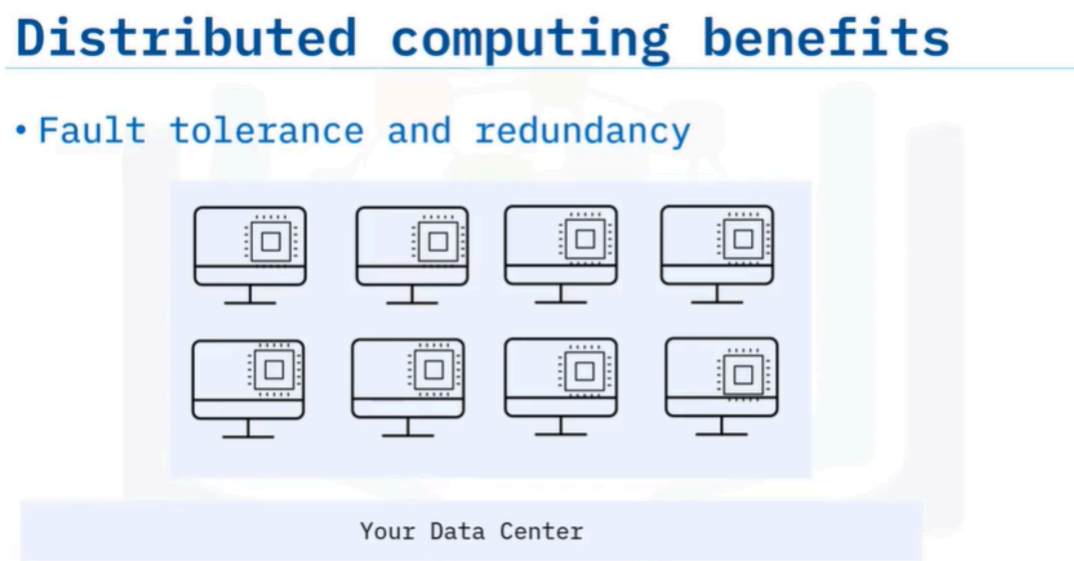
- People often use the terms Distributed computing and Parallel Computing interchangeably, as the two computing types have many similarities.



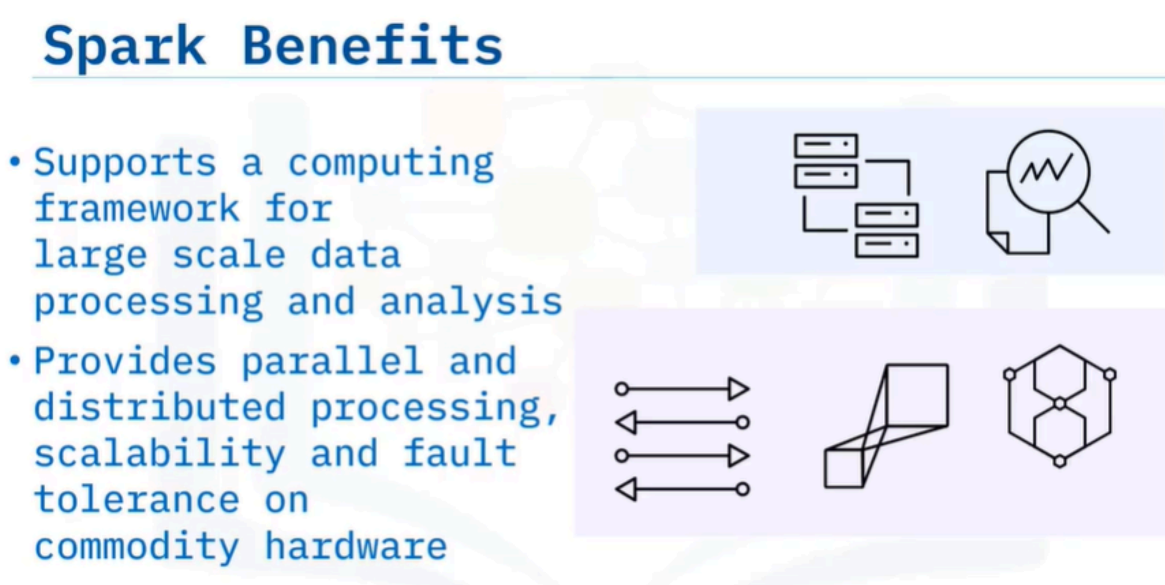
- However, distributed computing and parallel computing access memory differently. Typically, parallel computing shares all the memory, while in distributed computing, each processor accesses its own memory.



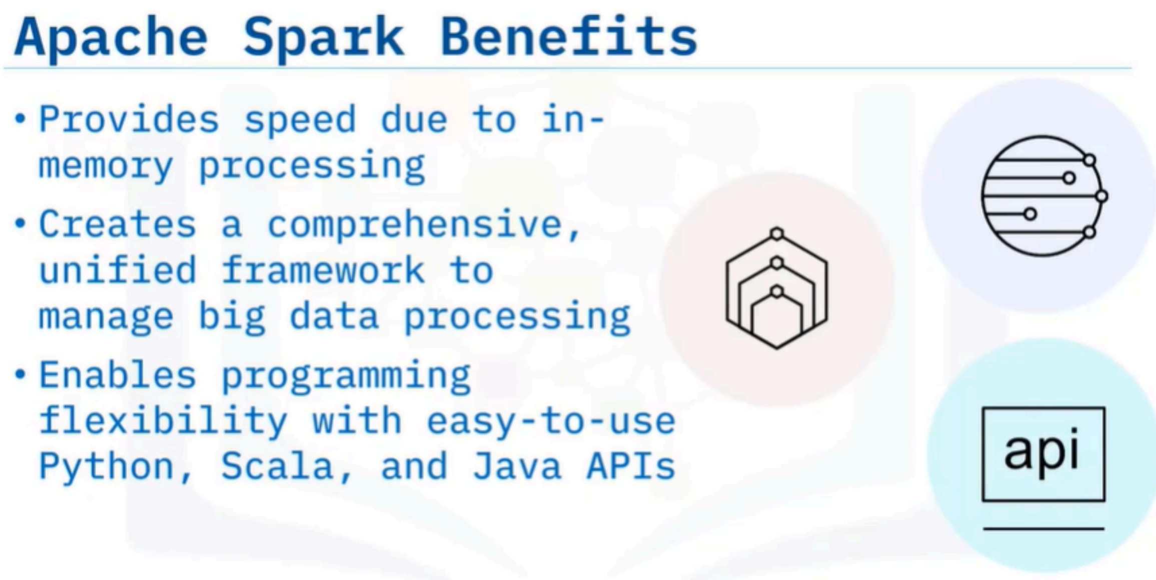
- Let's look at two top distributed computing benefits: First, distributed computing offers scalability and modular growth. Distributed systems are inherently scalable as they work across multiple machines and scale horizontally. A user can add additional machines to handle the increasing workload instead of repeatedly updating a single system, with virtually no cap for scalability.



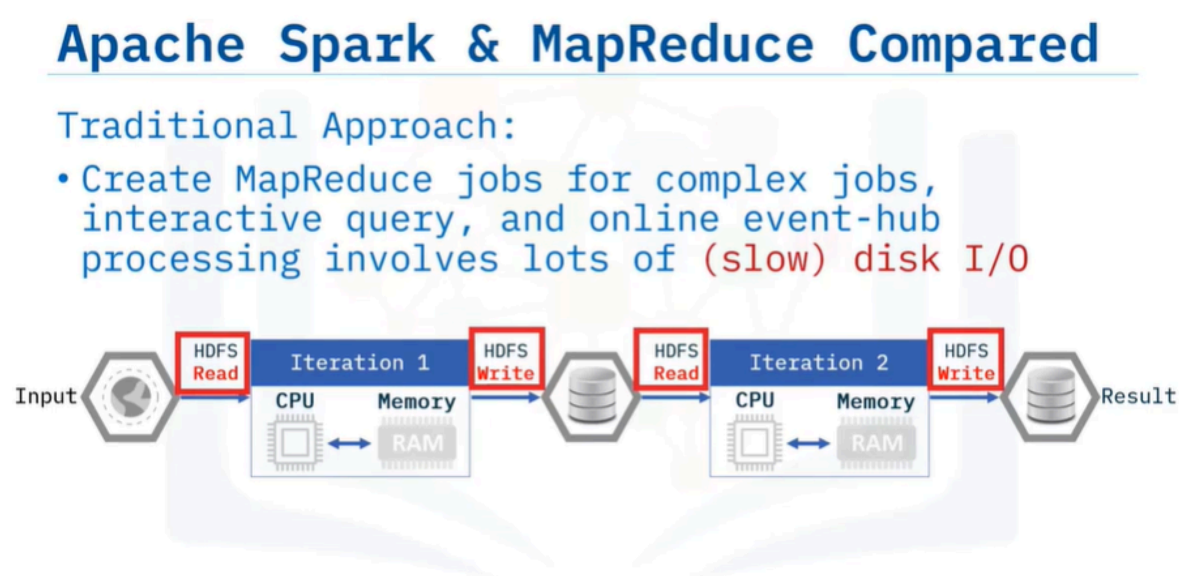
- Second, distributed systems require both fault tolerance and redundancy as they use independent nodes that could fail. Distributed computing offers more than fault tolerance. Distributed computing provides redundancy that enables business continuity. For example, a business running a cluster of eight machines can continue to function whether a single machine or multiple machines go offline.



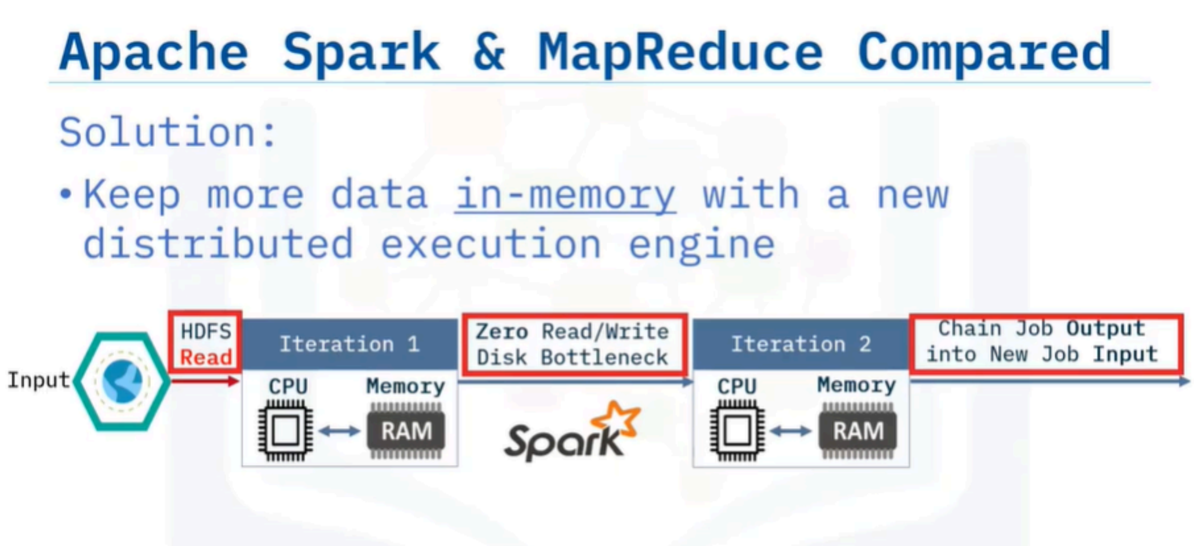
- Spark checks the boxes for all the benefits of distributed computing. Spark supports a computing framework for large-scale data processing and analysis. Spark also provides parallel distributed data processing capabilities, scalability, and fault-tolerance on commodity hardware.



- Spark provides in-memory processing and creates a comprehensive, unified framework to manage big data processing. Spark enables programming flexibility with easy-to-use Python, Scala, and Java APIs.

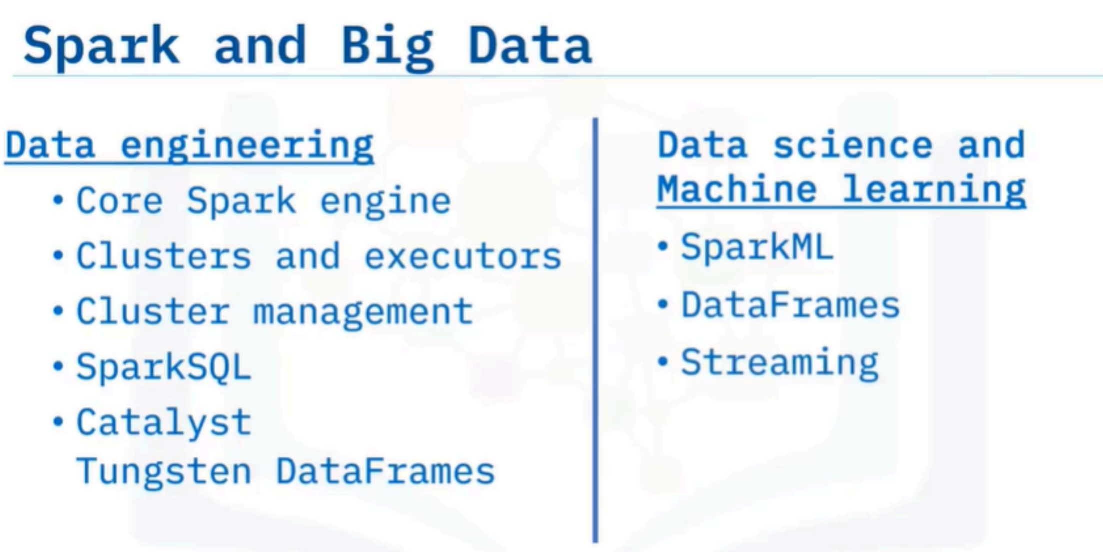


- How does Apache Spark compare to more traditional big data tools like MapReduce? A traditional MapReduce job creates iterations that require reads and writes to disk or HDFS. These reads and writes are usually time-consuming and expensive.



- Apache Spark solves the read/write problems encountered with MapReduce by keeping much of the data required in-memory and avoiding expensive disk

I/O, thus reducing the overall time by orders of magnitude.



- Spark provides big data solutions for both data engineering problems and data science challenges. Data engineers use Spark tools, including the core Spark engine, clusters and executors and their management, SparkSQL and DataFrames. Additionally, Spark also supports Data Science and Machine learning through libraries such as SparkML and Streaming.

Summary

In this video, you learned that:

- Spark is an open source in-memory application framework for distributed data processing and iterative analysis on massive data volumes
- Distributed computing is a group of computers or processors working together behind the scenes
- Both distributed systems and Apache Spark are inherently scalable and fault tolerant
- Apache Spark a large portion of the data required in-memory and avoids expensive and time-consuming disk I/O